AD23B31 - Image Processing and Computer Vision

Facial Emotion Recognition using Hybrid HOG LBP Feature Extraction and Machine Learning Classification

A MINI PROJECT REPORT

Submitted by

SHAKTHI G

March, 2026

RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering

Academic Year: 2025 – 2026

MINI PROJECT REPORT

On

Pedestrian Conflict Detection at Crosswalks using Computer Vision and Image Processing

Individual Report — Algorithm 2: Farneback Flow Velocity + Zone Check

Submitted by

SHAKTHI G

Roll No: 230701303

Section: CSE-E

Email: 230701303@rajalakshmi.edu.in

Date of Submission: [05-05-2026]

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background and Motivation

Facial emotion recognition is an important application of computer vision and artificial intelligence. It helps computers understand human emotions through facial expressions. Emotion detection systems are widely used in smart surveillance, healthcare, online learning, human-computer interaction, and security systems. Traditional emotion analysis methods require manual observation, which can be time-consuming and inaccurate. Therefore, automated real-time emotion recognition systems are becoming increasingly important.

This project uses OpenCV and DeepFace to detect human faces and identify emotions from webcam video in real time. The system captures video frames, detects faces, and predicts emotions such as happy, sad, angry, surprise, fear, neutral, and disgust.

1.2 Problem Statement

The project aims to build a real-time facial emotion recognition system using computer vision techniques. The system should automatically detect faces from webcam input and classify the emotions displayed by the detected faces.

1.3 Objectives

The objectives of this project are:

- To detect human faces using OpenCV Haar Cascade classifier.
- To analyze emotions using the DeepFace library.
- To display detected emotions in real time.
- To create a simple and efficient emotion monitoring system.

1.4 Scope of the Report

This project focuses on real-time emotion detection using webcam video input. The system performs face detection and emotion analysis without requiring complex training. It is mainly intended for academic and research purposes.

2. LITERATURE REVIEW

Facial emotion recognition has become an important research area in computer vision and deep learning. Researchers use facial analysis techniques to identify human emotions from images and videos.

2.1 Methods

Traditional methods for emotion recognition include feature extraction techniques such as Local Binary Patterns (LBP), Histogram of Oriented Gradients (HOG), and Support Vector Machines (SVM). Modern approaches mainly use deep learning and convolutional neural networks (CNNs) for higher accuracy.

2.2 Existing Studies

Several studies have used deep learning models for facial emotion recognition. DeepFace is widely used because it provides pre-trained deep learning models for facial analysis tasks including emotion detection, gender prediction, and face verification.

2.3 Gap Identified

Many deep learning systems require large datasets and long training times. This project addresses the problem by using a lightweight and ready-to-use DeepFace model with OpenCV for real-time emotion recognition.

3. DATASET DESCRIPTION

3.1 Dataset Source

The system uses live webcam video instead of a static dataset. Frames captured from the webcam are processed continuously for face detection and emotion analysis.

3.2 Emotion Categories

The detected emotions include:

- Happy
- Sad
- Angry
- Fear

- Surprise
- Neutral
- Disgust

3.3 Input Type

The project accepts real-time video frames captured through the system webcam.

4. PREPROCESSING METHODOLOGY

4.1 Frame Capture

Video frames are captured continuously using OpenCV VideoCapture.

```
cap = cv2.VideoCapture(0)
```

4.2 Grayscale Conversion

Frames are converted into grayscale format for efficient face detection.

```
gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

4.3 RGB Conversion

The grayscale image is converted into RGB format before emotion analysis.

```
rgb_frame = cv2.cvtColor(gray_frame, cv2.COLOR_GRAY2RGB)
```

4.4 Face Detection

Faces are detected using Haar Cascade classifier.

```
faces = face_cascade.detectMultiScale(gray_frame, scaleFactor=1.1, minNeighbors=5)
```

4.5 Face ROI Extraction

The detected face region is extracted for emotion analysis.

5. ALGORITHM: DEEPFACE + OPENCV

5.1 Algorithm Overview

The system combines OpenCV for face detection and DeepFace for emotion analysis. OpenCV detects faces from webcam frames, while DeepFace predicts the dominant facial emotion.

5.2 Face Detection using OpenCV

The Haar Cascade classifier detects faces from grayscale images.

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')
```

5.3 Emotion Detection using DeepFace

DeepFace analyzes the detected face and predicts the dominant emotion.

```
result = DeepFace.analyze(face_roi, actions=['emotion'], enforce_detection=False)
emotion = result[0]['dominant_emotion']
```

5.4 Output Display

The detected emotion is displayed on the video frame.

cv2.putText(frame, emotion, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)

5.5 Implementation Details

Parameter	Value
Face Detection	Haar Cascade
Emotion Model	DeepFace
Input Type	Webcam Video
Programming Language	Python
Libraries	OpenCV, DeepFace, TensorFlow
Output	Real-time Emotion Detection

6. RESULTS AND ANALYSIS

6.1 Output Results

The system successfully detected faces and predicted emotions in real time using webcam video.

6.2 Performance

The system works efficiently on standard systems with acceptable real-time speed.

6.3 Observations

- Happy and neutral emotions were detected accurately.
- Detection accuracy decreases under poor lighting conditions.
- Face detection performance depends on camera quality.
-

7. DISCUSSION

The project demonstrates that real-time emotion recognition can be implemented using simple computer vision techniques and pre-trained deep learning models. OpenCV provides fast face detection, while DeepFace offers accurate emotion classification.

The system is lightweight and easy to implement. However, the performance may decrease when faces are partially hidden or lighting conditions are poor. Despite these limitations, the project provides an effective solution for real-time emotion monitoring.

8. CONCLUSION AND FUTURE WORK

This project successfully implemented a real-time facial emotion recognition system using OpenCV and DeepFace. The system detects faces from webcam video and predicts emotions efficiently.

Future improvements include:

1. Using advanced face detection models for better accuracy.
2. Adding emotion tracking and analytics.
3. Deploying the system on mobile and embedded devices.
4. Improving performance under low-light conditions.

9. REFERENCES

1. OpenCV Documentation – https://docs.opencv.org/
2. DeepFace GitHub Repository – https://github.com/serengil/deepface
3. TensorFlow Documentation – https://www.tensorflow.org/
4. Viola, P., & Jones, M. Rapid Object Detection using a Boosted Cascade of Simple Features.

10. APPENDIX A – SOURCE CODE

```
import cv2
from deepface import DeepFace

# Load face cascade classifier
face_cascade = cv2.CascadeClassifier(
cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

# Start video capture
cap = cv2.VideoCapture(0)

while True:
ret, frame = cap.read()

gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
rgb_frame = cv2.cvtColor(gray_frame, cv2.COLOR_GRAY2RGB)

faces = face_cascade.detectMultiScale(
gray_frame,
scaleFactor=1.1,
minNeighbors=5,
minSize=(30, 30)
)

for (x, y, w, h) in faces:
face_roi = rgb_frame[y:y + h, x:x + w]

result = DeepFace.analyze(
face_roi,
actions=['emotion'],
enforce_detection=False
```

```
)

emotion = result[0]['dominant_emotion']

cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)

cv2.putText(frame, emotion, (x, y - 10),
cv2.FONT_HERSHEY_SIMPLEX,
0.9, (0, 0, 255), 2)

cv2.imshow('Real-time Emotion Detection', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
break

cap.release()
cv2.destroyAllWindows()
```